

How Mathitude Handles Card Data

Ari · for Paula · 2026-05-24
May 24, 2026

CONTENTS

How Mathitude Handles Card Data	
In one sentence	
How card data moves (zoomed in)	
Full system data flow	
What Mathitude has (and what Mathitude doesn't)	
We have	
We do not have	
Data model — one card per parent	
Access controls	
Two admin tiers	
Stripe API key	
Webhooks	
Logs	
Questions Paula might be asked	
SOC 2 path (if and when Mathitude wants its own attestation)	
Sign-off checklist	

How Mathitude Handles Card Data

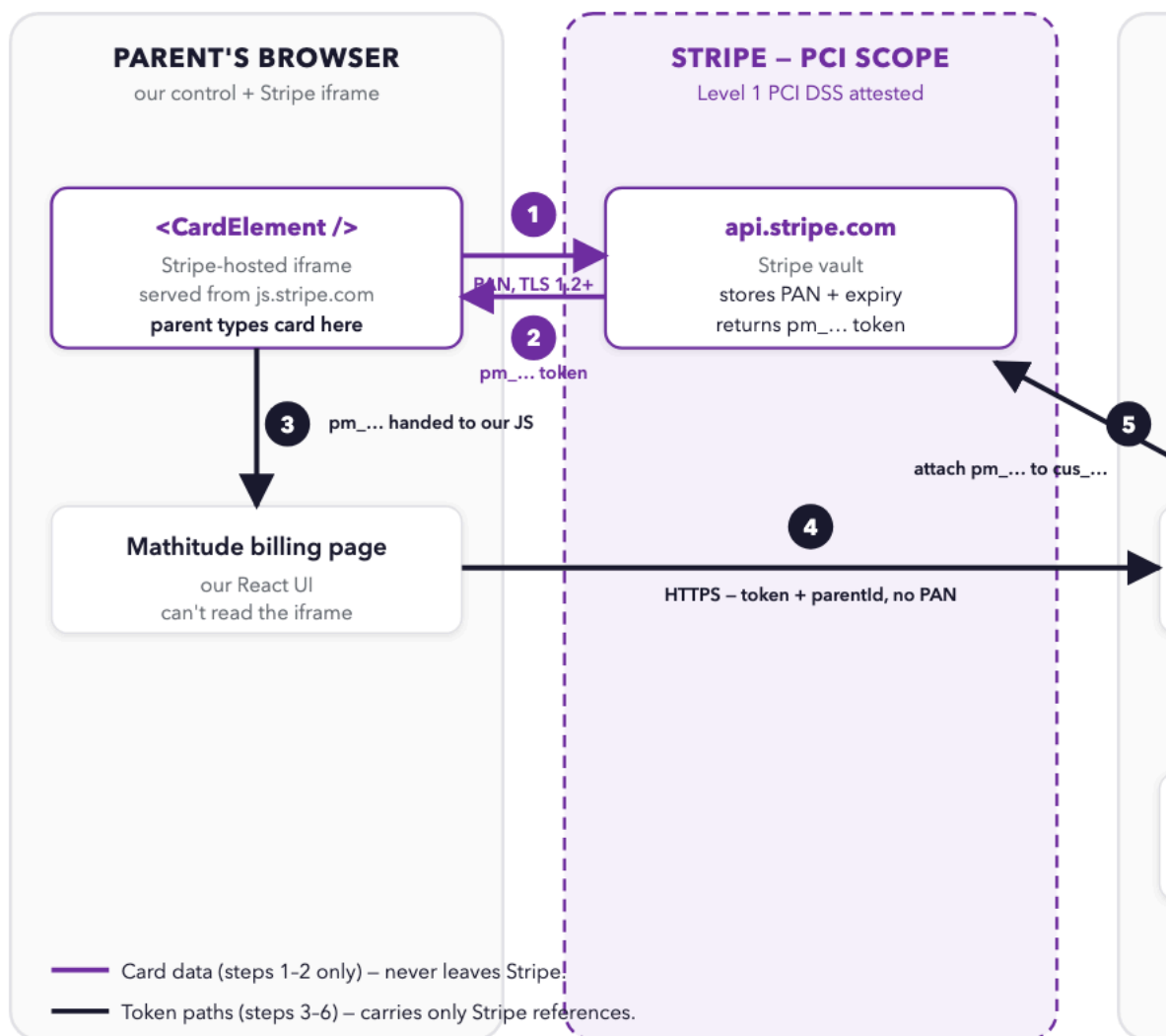
A plain-English compliance brief for Paula to forward. Updated 2026-05-24. Author: Ari (ari@coframe.com).

In one sentence

Mathitude never sees, transmits, or stores credit card numbers. Cards go from the parent's browser **directly to Stripe** inside a Stripe-hosted iframe. Mathitude only ever holds Stripe's reference tokens (`pm_...` , `cus_...`) and a card's brand + last 4 digits for display.

Stripe is **PCI DSS Level 1** — the highest tier of card-handler compliance, the same category as Apple Pay and large banks.

How card data moves (zoomed in)

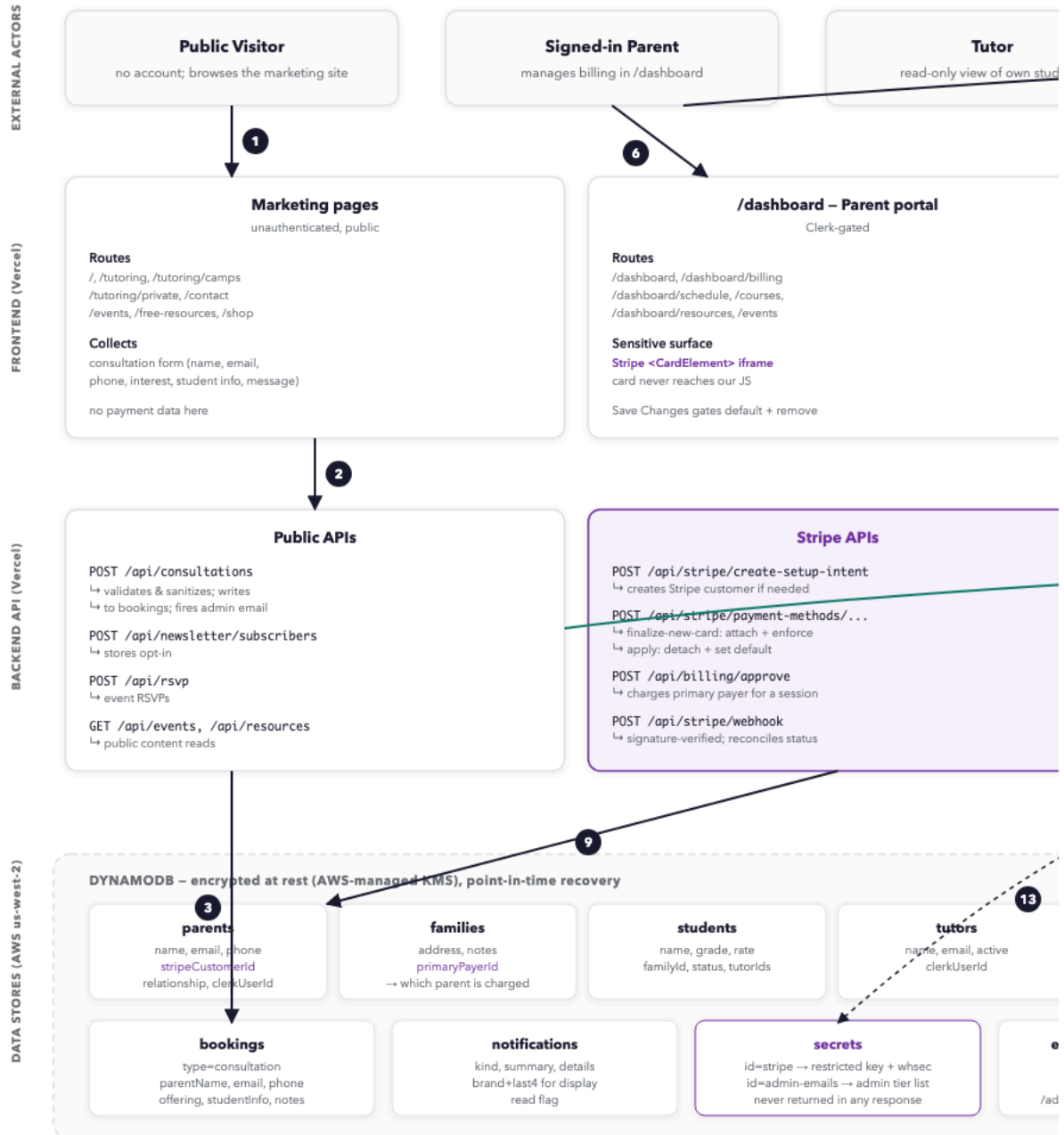


Full system data flow

A wider view: every data store, every API surface, every external service, and which arrow carries what. The card-flow above zooms into hops 7–9 of this diagram.

Mathitude – System Data Flow

Where every piece of data lives, how it moves, and which trust boundaries it crosses.



The card number lives inside the dashed purple zone — never outside. Steps 1 and 2 happen between Stripe's iframe and Stripe's servers; our JavaScript and our backend are not on those hops. Steps 3, 4, 5, 6 carry **only** Stripe's reference tokens.

What Mathitude has (and what Mathitude doesn't)

WE HAVE

Data	Sensitivity	Where it lives
Stripe customer reference (<code>cus_...</code>)	Non-sensitive	DynamoDB <code>parents</code> table
Stripe payment method reference (<code>pm_...</code>)	Non-sensitive	DynamoDB
Card brand (Visa, Mastercard, ...)	Non-sensitive	DynamoDB <code>payments</code> + display only
Last 4 digits of card	Non-sensitive (per PCI DSS)	DynamoDB + display only
Card expiry month / year (for display)	Non-sensitive	DynamoDB + display only
Charge history (amount, status, date)	Business data	DynamoDB <code>payments</code>

WE DO NOT HAVE

- Full card number (PAN)
- CVV / CVC
- Cardholder name (Stripe holds it)
- Billing ZIP (Stripe holds it)
- Anything that could be used to charge the card without Stripe

These remain on Stripe's PCI-compliant infrastructure. We could not hand them to a hacker, a subpoena, or a curious bookkeeper because we do not possess them.

Data model — one card per parent

Each parent on a family has their own Stripe customer with exactly one card on file. A family can have multiple parents, so a family can have multiple cards available. The family record points at one parent as the **primary payer** — that's whose card gets charged. Switching the primary payer is a one-click action; the system charges the new parent's card from the next billing event onward.

This matches real-world households: husband adds his card under his parent record, wife under hers, and Paula picks whose card runs at billing time.

Access controls

TWO ADMIN TIERS

- **Master admin** — full control, including adding/removing other admins. Bootstrap master admins (Paula, Ari, Nikki) are hard-coded and cannot be removed via the UI.
- **Admin** — full operator access to families, students, sessions, billing, payments, financials. Cannot manage other admins; if a plain admin tries to mutate the admin list, the server returns 403.

Both tiers sign in through Clerk (a SOC 2 Type 2 attested identity provider). There is no admin path that doesn't go through Clerk.

STRIPE API KEY

We use a Stripe **restricted key** (`rk_...`) scoped to the four permissions we actually need: payment methods, customers, payment intents, setup intents. The key is stored encrypted at rest in DynamoDB and is never returned in any HTTP response — only the last 4 characters appear in the diagnostic display under Settings → Stripe.

WEBHOOKS

Every incoming Stripe webhook event is verified against our `STRIPE_WEBHOOK_SECRET` before being acted on. Forged or stale-timestamped events are rejected with HTTP 400. This prevents a third party from sending fake “payment succeeded” events to mark debts as paid.

LOGS

`grep -r "cardNumber\|card_number\|pan" src/` returns zero hits across the codebase. Application logs (Vercel + AWS CloudWatch) contain no PANs.

Questions Paula might be asked

Where is my card stored? On Stripe’s servers, the same infrastructure used by Lyft, Shopify, Substack, Slack, and tens of thousands of other businesses. Stripe is PCI DSS Level 1 attested — the top compliance tier. Mathitude has a reference to your card, not the card itself.

Can a Mathitude employee see my card number? No. The card number never reaches Mathitude’s servers, and Mathitude’s JavaScript can’t read inside Stripe’s input field. The strongest claim isn’t a policy; it’s that we don’t have the data to look at.

What if Mathitude’s database gets hacked? An attacker would see Stripe reference tokens (which require Mathitude’s restricted Stripe key to use) and the last 4 digits of cards. They would not see card numbers, expiries, or CVCs because those are not in the database. The Stripe key itself is encrypted at rest with an AWS-managed key; access logs would show any read.

What if Stripe gets hacked? Then everyone who uses Stripe (including most of the internet) has a problem. Stripe carries Level 1 PCI attestation precisely to make this extremely unlikely.

How do you charge me without seeing my card? We call Stripe’s API and say: “Charge \$100 from customer `cus_xyz` using payment method `pm_abc`.” Stripe handles the actual card transaction. Our request never includes a card number.

Can I get a copy of this? Yes — Paula has the PDF version. If you want a fresh dated copy, ask her.

SOC 2 path (if and when Mathitude wants its own attestation)

Stripe, Clerk, AWS, Vercel — all of Mathitude’s sub-processors carry their own SOC 2 attestations. Most of the controls a SOC 2 auditor would check for Mathitude are inherited from those vendors.

The Mathitude-side work is access logging, encryption verification, vendor management, and an incident-response runbook. Realistic cost: ~\$10–15K/year for a compliance vendor (Vanta, Drata, Secureframe), ~3 months for Type 1 attestation, ~9 months for Type 2.

Cheapest first step today: enable AWS CloudTrail with 365-day retention

- GuardDuty. <\$50/month, required evidence for any future audit.
-

Sign-off checklist

Question	Answer
Does Mathitude see card numbers?	No.
Does Mathitude transmit card numbers to its own servers?	No.
Does Mathitude store card numbers?	No — only Stripe references.
Is Stripe PCI-compliant?	Yes, Level 1 (highest tier).
Are admin actions authenticated?	Yes, via Clerk.
Are admin mutations restricted to master admin?	Yes.
Are Stripe webhooks signature-verified?	Yes.
Is the database encrypted at rest?	Yes, AWS KMS.
Can a Mathitude breach leak card data?	No — we don't have it to leak.

Maintainer: Ari (ari@coframe.com). Latest version always at `COMPLIANCE.md` in the Mathitude repository.